

Indexing in MongoDB: how a database finds one row in a crore

The problem indexes solve, what an index really is on disk, why almost every database reaches for a B+ tree, every MongoDB index type with a step-through animation, and a decision guide for what to use when.

[Cohort · Backend](#)[B+ tree insert / delete](#)[10 index types, animated](#)[explain\(\) · ESR rule](#)

```
db.users.find({ email: "riya@x.com" })
```

without index → scan 10,00,000 docs ~ 1200 ms

with index → read 3 tree pages ~ 0.4 ms

3000× faster. Same query. Same data.

The only thing that changed is the
data structure the query walks.

01 The problem: finding one document in ten lakh

Before the solution, feel the pain that created it.

You built a login route. It works perfectly in class with 50 test users. Then the product launches, the **users** collection crosses ten lakh documents, and the same route starts taking more than a second. Nothing in your code changed. What changed is *how much data the query has to walk through*.

Watch what MongoDB actually does when there is no index and you ask for one email:

```
db.users.find({ email: "sneha@x.com" })
```

d1

email: **riya**@x.com
age: 22

d2

email: **aman**@x.com
age: 27

d3

email: **neha**@x.com
age: 24

d4

email: **karan**@x.com
age: 31

d5

email: **pooja**@x.com
age: 22

d6

email: **rahul**@x.com
age: 29

d7

email: **sneha**@x.com
age: 26

d8

email: **vikas**@x.com
age: 35

DOCS EXAMINED

0

MATCHES FOUND

0

STAGE

—

Eight documents in **users**, stored in the order they were inserted. No index exists on **email**. The query asks for one email. Where does MongoDB start?

THE REAL COST IS NOT THE COMPARISON

A CPU can compare ten lakh strings in a few milliseconds. The expensive part is **reading the documents from disk** so they can be compared. Disks (even SSDs) serve data in fixed-size blocks called **pages**, and a full scan drags every page of the collection through memory. The problem is I/O, not CPU — and that is why the fix is a data structure, not a faster loop.

Why 'just add more RAM' is not the answer

MongoDB's storage engine keeps a cache in RAM, so a small collection that fits entirely in memory scans fast. But a scan still touches *every* document, so CPU time and cache pressure still grow linearly. And the moment the collection outgrows the cache — which happens to every successful product — each scan turns into

thousands of disk reads. Linear cost is the enemy; we need a lookup whose cost barely grows as the data grows.

The everyday version of this problem

A railway station displays a **coach position board**: coach S7 stops at platform marker 9. Without it, you would walk along the entire 24-coach train reading numbers on every coach — a collection scan. With it, you read one small board and walk straight to marker 9. The board is not the train; it is a small, separately maintained structure that tells you *where* to look. That is exactly what an index is.

02 What an index is

A separate, sorted, pointer-carrying copy of one field — kept in sync on every write.

An index is **not** a setting that makes a collection faster. It is a **second data structure** that MongoDB builds and stores on disk next to the collection: the values of one or more fields, kept in **sorted order**, each paired with a **pointer** to the document that owns it.

IXSCAN + FETCH — index on email

← → keys also work

```
db.users.find({ email: "sneha@x.com" })
```

INDEX ON { EMAIL: 1 } — SORTED KEYS → POINTERS

aman@x.com → d2

karan@x.com → d4

neha@x.com → d3

pooja@x.com → d5

rahul@x.com → d6

riya@x.com → d1

sneha@x.com → d7

vikas@x.com → d8

COLLECTION (INSERTION ORDER)

d1

email: **riya@x.com**

age: 22

d2

email: **aman@x.com**

age: 27

d3

email: **neha@x.com**

age: 24

d4

email: **karan@x.com**

age: 31

d5

email: **pooja@x.com**

age: 22

d6

email: **rahul@x.com**

age: 29

d7

email: **sneha@x.com**

age: 26

d8

email: **vikas@x.com**

age: 35

INDEX KEYS EXAMINED

0

DOCS EXAMINED

0

STAGE

—

Same 8 documents, same query. But now an **index on email** exists: a separate, **sorted** list of every email, each with a pointer to its document. The collection itself is untouched and still unordered.

DEFINITION

An **index** is an ordered mapping from **indexed value → location of the document**, stored separately from the documents and updated automatically on every insert, update and delete. Because it is ordered, the database can find a value without reading everything, and can return documents in that order without sorting them.

Three things every index gives you

● Fast equality lookups

`find({ email: "x" })` — go straight to the matching key instead of scanning.

● Fast range scans

`find({ age: { $gte: 18, $lt: 25 } })` — find the first matching key, then walk forward through sorted neighbours until the range ends.

● Free ordering

`sort({ createdAt: -1 })` — the keys are already sorted, so MongoDB just reads them backwards. Without an index, a sort over a large result set happens in memory and can hit the 100 MB limit.

The price: every write now does extra work

The index must stay correct. When you `insertOne()` a user, MongoDB writes the document *and* inserts a key into every index on that collection. Update the email? The old key is removed and a new one inserted. Delete the user? Every index loses an entry. An index trades **slower writes and more disk/RAM** for **faster reads**. It is a trade, not free speed.

indexes on the collection



indexes: 3 → 4 structures updated per insert

One `insertOne()` = 1 document write + 3 index-entry writes = 4 B+ tree updates, every one of which may split a page. Reads get faster; writes pay for it.

 **Every collection already has one index.** MongoDB automatically creates a unique index on `_id`. That is why `findById()` in Mongoose is always fast, and why you have never had to think about it.

03 Why databases use B+ trees

The disk works in pages. The B+ tree is the data structure that respects that.

The last section showed the idea: keep keys sorted, search cleverly. But 'sorted' is not enough on its own — the shape of the structure has to match how disks and memory actually behave. This is where students usually get told 'MongoDB uses B-trees' and nothing more. Let's see *why*.

Sorted array vs B+ tree: count the page reads

← → keys also work

lookup 51 — each page = one disk read

SORTED ARRAY ON DISK — 16 KEYS, 4 PER PAGE

page 1

5

8

12

15

page 2

20

23

27

30

page 3

33

37

40

44

PAGES READ (ARRAY)

0

PAGES READ (B+ TREE)

—

A sorted list already lets us binary-search. So why not just store index keys as one big sorted array on disk? Because the disk does not give you 'one key' — it gives you a **whole page** (typically 4–32 KB) at a time. Here: 16 keys, 4 per page.

The four candidates, and why three of them lose

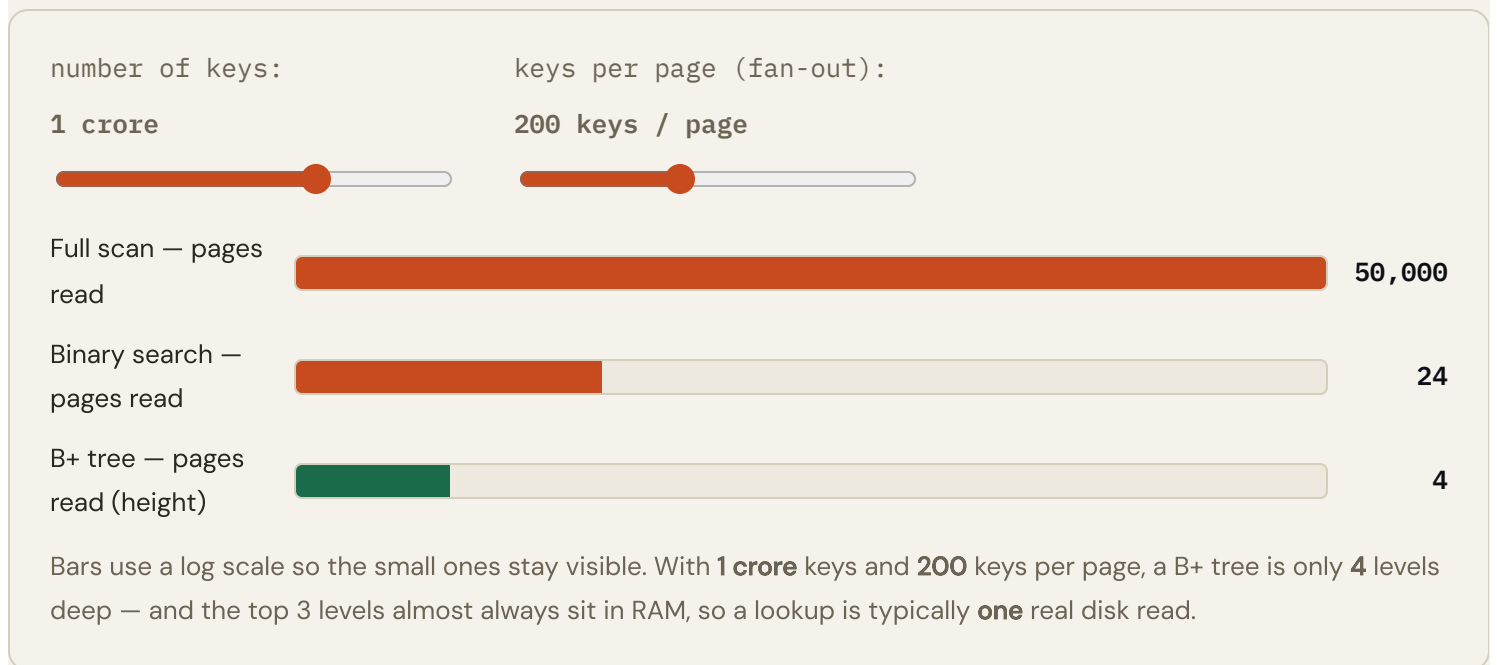
Structure	Lookup	Range / sort	Insert in the middle	Verdict for a disk index
Sorted array	$\log_2 n$ comparisons	excellent — keys are adjacent	terrible — shift everything after it	Fine for read-only data; inserts kill it
Hash table	$O(1)$ average	none — neighbours are scattered	fine, until a resize rehashes everything	Only equality. No <code>\$gt</code> , no <code>sort()</code>
Balanced BST (AVL / red-black)	$\log_2 n$	good (in-order walk)	good	Each node holds 1 key → 1 tiny node per page read. A crore of keys = ~24 levels = ~24 disk reads.

Structure	Lookup	Range / sort	Insert in the middle	Verdict for a disk index
B+ tree	$\log_f n$ — f is hundreds	excellent — leaves are linked	good — local split, rest untouched	One node = one page, 3–4 levels for billions of keys. The winner.

THE INSIGHT IN ONE SENTENCE

A binary tree asks *one* question per disk read. A B+ tree packs *hundreds* of questions into the same disk read, so it needs far fewer reads. Fan-out is the whole trick.

How flat is 'flat'? Try it.



B-tree vs B+ tree — the difference that matters for range queries

Both are wide, balanced, page-sized trees. The difference is *where the data lives*. In a **B-tree**, every node stores keys *and* their values, so a key you are looking for can be found in an internal node and the search stops early. In a **B+ tree**, internal nodes hold *only* routing keys; all the real key → pointer pairs live in the **leaves**, and the leaves are **linked together** in sorted order.

That second design wins for databases because (1) internal nodes without values can hold *more* keys per page, so the tree is even flatter, and (2) a range query like **age between 18 and 25** becomes: find the first leaf, then *walk the chain* — no bouncing back up through the tree.

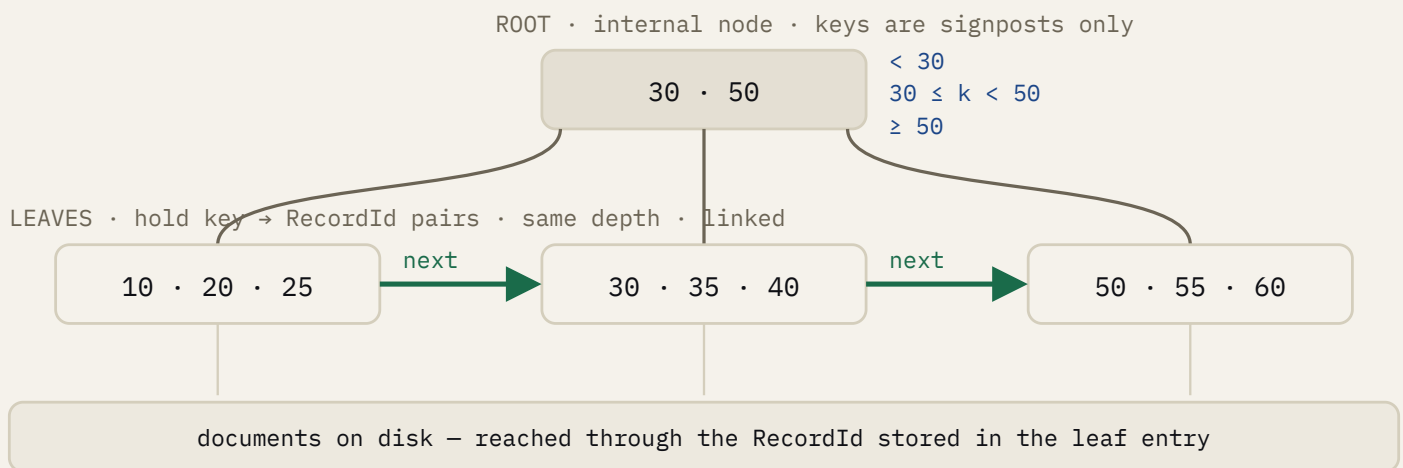
💡 **What MongoDB actually uses.** The default storage engine, **WiredTiger**, stores both collections and indexes as B-trees whose *internal pages hold only keys and child references* and whose *leaf pages hold the key/value pairs* — i.e. the B+ tree layout described here. The MongoDB docs still say 'B-tree'; the layout is the one that matters, and that layout is B+.

04 B+ tree: exact working, insert and delete

The same rules your DBMS textbook uses (Korth), animated step by step.

You now know *why* the B+ tree wins. This section is the *how*: the rules a node obeys, how a key is added, how one is removed, and how the tree keeps itself balanced without anyone rebuilding it. Every walkthrough below is computed live by a real B+ tree implementation — not hand-drawn frames — so what you see is what the algorithm does.

Anatomy and the rules of the game



order = max children per node · here max 3 keys / 4 children · every leaf at the same depth = balanced

RULES FOR THIS DEMO (ORDER 4 — MAX 3 KEYS PER NODE)

Every node holds at most **3** keys. A node with 4 keys has **overflowed** and must split.

Leaf nodes must keep at least **2** keys ($\lceil 3/2 \rceil$). **Internal nodes** must keep at least **1** key, i.e. 2 children ($\lceil 4/2 \rceil$). Falling below the minimum is **underflow**. The root is exempt from minimums.

Internal keys are signposts: the pointer to the left of key k leads to values $< k$; the pointer to its right leads to values $\geq k$. Real data (key → RecordId) lives only in leaves, which are linked in sorted order.

💡 In a real index a node is a whole disk page holding hundreds of keys, so splits and merges are rare. We use 3 keys per node so that you can see them happen within a dozen inserts. The algorithm is identical.

Insert: search down, place, split up

Insertion always has the same three beats. **Search** from the root to the one leaf where the key belongs. **Place** the key in sorted position. If the leaf overflows, **split** it in half and push a separator up — and if *that* overflows the parent, split again, all the way up. The tree only gets taller when the root itself splits, which is why every leaf is always at the same depth.

Insert in this order and watch for two leaf splits, then an internal split that creates a new root: 10 20 30 40 25 50 60 35 70 80 55 15.

B+ tree insert – 12 keys, 55 steps

jump to: insert 10 ▼ · insert 10

ROOT (LEAF)



height 1 keys 0 max keys/node 3 min keys: leaf 2, internal 1

☐ visited / compared ☒ new node ☐ overflow / underflow ☐ sibling lending

Start at the **root** []. We are looking for the leaf where **10** belongs.

⚠️ **Common exam mistake.** On a *leaf* split the separator is **copied** up (it stays in the leaf). On an *internal* split the middle key **moves** up (it leaves the node). Students who mix these up produce trees where a real key vanishes or a signpost is duplicated.

Delete: remove, then borrow or merge

Deletion is where B+ trees earn their reputation for being tricky, but it is only four cases. Remove the key from its leaf. If the leaf still meets the minimum, **done**. If not, **borrow** a key from an adjacent sibling that has spare keys and fix the separator in the parent. If neither sibling can spare one, **merge** with a sibling and delete the separator from the parent — which may make the *parent* underflow, so repeat one level up. If the root ends up with zero keys, drop it: the tree gets shorter.

Starting from the tree the inserts built, delete `30 → 80 → 20 → 50`. Each one triggers a different case, in order of difficulty.

B+ tree delete – 4 keys, 4 cases, 33 steps

jump to:

delete 30 ▼

 · delete 30

ROOT

INTERNAL

LEAVES

50 | . | .

20 | 30 | .

70 | . | .

10 | 15 | .

20 | 25 | .

30 | 35 | 40

50 | 55 | 60

70 | 80 | .

height 3

keys 12

max keys/node 3

min keys: leaf 2, internal 1

visited / compared

new node

overflow / underflow

sibling lending

Start at the root [50]. We are looking for the leaf where 30 belongs.

Case	Trigger	What happens	Parent separator
1 • Simple	leaf still has $\geq$ min keys	remove key, stop	untouched — even if it equals the deleted key (it becomes a signpost)
2 • Borrow (redistribute)	leaf underflows, a sibling has $>$ min keys	sibling hands over its nearest key	rewritten to the first key of the right-hand leaf
3 • Merge leaves	leaf underflows, both siblings at min	two leaves become one	deleted from parent (it was only a copy)
4 • Merge internal / shrink	the parent itself underflows after a merge	internal nodes merge; if root empties, its child becomes root	pulled down into the merged node (internal keys are not copies)

Free play: build your own tree

Type any number. The same code that drew the walkthroughs animates your operation, one rule at a time. Try inserting `1..9` in order to see a right-leaning fill, or delete keys until the root collapses.

B+ tree lab

animation plays automatically after each operation

key

Insert

Delete

Insert random

max keys/node3

Load sample tree

Clear

ROOT

INTERNAL

LEAVES

height 3 keys 12 max keys/node 3 min keys: leaf 2, internal 1

Loaded the sample tree from the walkthroughs. Type a number and press **Insert** or **Delete** — every step of the algorithm plays out here.

05 Inside MongoDB: WiredTiger, RecordId and the two trees

What actually happens between `find()` and the bytes on disk.

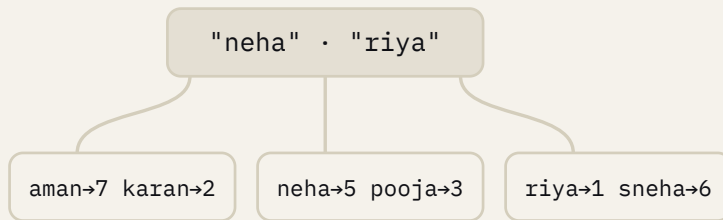
Students often picture 'the index points at the document' as a memory address. In MongoDB it is one B+ tree pointing into another. Seeing both trees makes `explain()` output obvious later.

IXSCAN → FETCH: two B+ tree walks

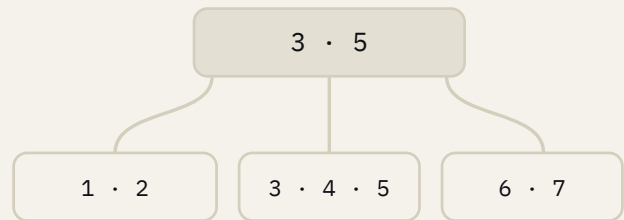
← → keys also work

```
db.users.find({ email: "karan@x.com" })
```

INDEX B+ TREE { email: 1 } — key → RecordId



COLLECTION B+ TREE — RecordId → BSON document



(collection not touched yet)

STAGE

—

KEYS EXAMINED

0

DOCS EXAMINED

0

In WiredTiger **everything is a B+ tree** — including the collection itself. The collection tree is keyed by **RecordId** (an internal 64-bit number). An index tree is keyed by the indexed value, and its leaf entries store the RecordId of the owning document.

Index keys are stored in a canonical, comparable form

WiredTiger cannot compare raw BSON fast enough, so MongoDB encodes each index key into a **KeyString**: a byte string designed so that plain `memcmp` ordering equals BSON ordering (numbers before strings, arrays element-wise, and so on). This is why a compound key `{ city: 1, age: -1 }` can be stored in one B+ tree — the two fields are concatenated into one sortable byte string, with the sort direction baked into the encoding.

Where the cache fits

WiredTiger keeps a cache (by default about half of RAM minus 1 GB) of recently used pages from both trees. A page not in cache is read from disk. Practically: the top levels of every hot index live in RAM permanently, leaf pages come and go, and documents are the most likely thing to miss. That ordering is why the **working set** advice exists — your *indexes* should fit in RAM, even when your data cannot.

RECORDID VS _ID

`_id` is a user-visible field with its own unique index. **RecordId** is WiredTiger's internal key for the collection tree. `find({ _id: x })` walks the `_id` index to get a RecordId, then walks the collection tree — exactly like any other indexed query. (Clustered collections, an opt-in feature, make `_id` the RecordId itself and skip the second walk.)

06 Creating indexes and reading explain()

Syntax, then the three numbers that tell you whether an index is working.

MONGOSH — CREATE, LIST, DROP

Copy

```
// single field, ascending. The name becomes "email_1"
db.users.createIndex({ email: 1 })

// with options
db.users.createIndex({ email: 1 }, { unique: true, name: "uniq_email" })

// see every index on the collection (always includes _id_)
db.users.getIndexes()

// remove one
db.users.dropIndex("uniq_email")

// total on-disk size of all indexes for this collection
db.users.totalIndexSize()
```

MONGOOSE — INDEXES LIVE IN THE SCHEMA

Copy

```
const userSchema = new mongoose.Schema({
  email: { type: String, required: true, index: true }, // simple
  phone: { type: String, unique: true },                // unique index
  city: String,
  age: Number,
});

// compound index — declared on the schema, not the field
```

```
userSchema.index({ city: 1, age: -1 });
```

```
// Mongoose calls createIndex() for each of these when the model is
// compiled (autoIndex). In production, turn autoIndex off and create
// indexes deliberately – builds on big collections are not free.
mongoose.connect(uri, { autoIndex: process.env.NODE_ENV !== "production" });
```

Reading `explain("executionStats")`

Every performance conversation about MongoDB comes down to one command. Run the query with `.explain("executionStats")` and read three things: the **stage names**, **totalKeysExamined**, and **totalDocsExamined**.

MONGOSH – EXPLAIN OUTPUT (TRIMMED TO WHAT MATTERS)

Copy

```
{
  "queryPlanner": {
    "winningPlan": {
      "stage": "FETCH", // step 2: read docs
      "inputStage": {
        "stage": "IXSCAN", // step 1: walk the index
        "keyPattern": { "email": 1 },
        "indexName": "email_1",
        "isMultiKey": false,
        "direction": "forward",
        "indexBounds": { "email": [ "[\"sneha@x.com\", \"sneha@x.com\"]" ] }
      }
    },
    "rejectedPlans": []
  },
  "executionStats": {
    "nReturned": 1, // docs returned
    "executionTimeMillis": 0,
    "totalKeysExamined": 1, // index entries touched
    "totalDocsExamined": 1 // documents touched
  }
}
```

FIELD	WHAT IT MEANS	HEALTHY LOOKS LIKE
<code>winningPlan.stage</code>	the tree of stages the planner chose	IXSCAN at the bottom; COLLSCAN means no usable index
<code>totalKeysExamined</code>	index entries the engine looked at	close to <code>nReturned</code>
<code>totalDocsExamined</code>	documents read from the collection	close to <code>nReturned</code> , or <code>0</code> for a covered query
<code>indexBounds</code>	the key range the IXSCAN walked	tight bounds; <code>[MinKey, MaxKey]</code> on a field means it was not really used
<code>rejectedPlans</code>	other candidate plans the planner tried	empty or short; many rejected plans hints at too many overlapping indexes
<code>stage: SORT</code>	an in-memory sort happened	absent — the index should provide the order

THE RATIO THAT MATTERS

totalDocsExamined ÷ nReturned. If a query returns 10 documents but examined 1 lakh, the index is not selective for that query (or is missing). Aim for a ratio near 1. This single number catches most real-world slow queries.

Index builds and hidden indexes

Since MongoDB 4.2, all index builds are 'hybrid': the collection stays readable and writable while the index is built, with a brief lock at the start and end. On a big collection a build still costs CPU and I/O, so schedule it. Before dropping an index you are not sure about, **hide** it: the planner stops using it but it is still maintained, so un-hiding is instant if things get slower.

MONGOSH — HIDE BEFORE YOU DROP

Copy

```
db.users.hideIndex("city_1_age_-1") // planner ignores it; still updated on writes
// watch the app for a day...
db.users.unhideIndex("city_1_age_-1") // instant rollback
// or, if nothing slowed down:
db.users.dropIndex("city_1_age_-1")

// which indexes are actually used? (ops = times used since server start)
db.users.aggregate([{$indexStats: {}}])
```

07 Every index type, animated

Same B+ tree underneath. What changes is what becomes the key.

Here is the secret that makes this section easy: **every one of these is the same B+ tree**. The type only decides *what string of bytes becomes the key* — one field, two fields glued together, each element of an array, a stemmed word, a map cell id, a hash. Once you see that, the behaviours and limits of each type stop being trivia and start being obvious.

Single fieldCompoundMultikeyTextGeospatialHashedUniqueSparse

PartialTTLWildcard

Type	Key that goes in the tree	Kind
Single field	the field value	structure
Compound	field ₁ + field ₂ + ... concatenated	structure
Multikey	one entry per array element	automatic
Text	each stemmed word (+ weight)	structure
Geospatial (2dsphere)	S2 cell ids of the location	structure
Hashed	hash(value)	structure
Wildcard	path name + value, for every path	structure
Unique	(same tree) — rejects duplicate keys	property
Sparse	(same tree) — skips docs missing the field	property
Partial	(same tree) — only docs matching a filter	property
TTL	(same tree on a date) — background deletes	property
Hidden	(same tree) — maintained, ignored by the planner	property

7.1 • Single-field index

The default. One field, one direction, every lookup / range / sort on that field.

Single field – build, lookup, range, sort

← → keys also work

```
db.users.createIndex({ age: 1 })
```

USERS — AS STORED

d1

name: **riya**

age: 22

d2

name: **aman**

age: 27

d3

name: **neha**

age: 24

d4

name: **karan**

age: 31

d5

name: **pooja**

age: 22

d6

name: **rahul**

age: 29

INDEX { AGE: 1 } — KEY → RECORDID

The simplest index: one field, one direction. **1** means ascending, **-1** descending. For a single-field index the direction does not matter for lookups — the tree can be read either way.

7.2 • Compound index

Several fields in one key. Order in the definition decides which queries it can serve.

Compound — the prefix rule

← → keys also work

```
db.users.createIndex({ city: 1, age: 1 })
```

USERS

d1

d2

city: Bhopal
age: 22

city: Indore
age: 27

d3
city: Bhopal
age: 24

d4
city: Pune
age: 31

d5
city: Indore
age: 22

d6
city: Delhi
age: 29

INDEX { CITY: 1, AGE: 1 } — ONE KEY MADE OF TWO FIELDS

Bhopal • 22 → d1

Indore • 27 → d2

Bhopal • 24 → d3

Pune • 31 → d4

Indore • 22 → d5

Delhi • 29 → d6

PLAN

A compound index sorts by the **first** field, and within equal values of the first, by the **second** — like a phone book sorted by surname, then first name. Field *order* in the definition is everything.

 **Direction matters for sorts, not lookups.** { city: 1, age: -1 } can serve sort({ city: 1, age: -1 }) and its exact reverse sort({ city: -1, age: 1 }) ,but **not** sort({ city: 1, age: 1 }) .When you design a compound index for a sort, copy the sort's directions.

7.3 • Multikey index

What happens when the indexed field is an array.

Multikey — arrays explode into entries

← → keys also work

```
db.posts.createIndex({ tags: 1 }) // auto-multikey
```

POSTS – TAGS IS AN ARRAY

p1

title: Node basics

tags: [node, js, backend]

p2

title: React hooks

tags: [react, js]

p3

title: Docker 101

tags: [docker, devops]

p4

title: Express auth

tags: [node, auth, backend]

MULTIKEY INDEX { TAGS: 1 } – ONE ENTRY PER ARRAY ELEMENT

You do not *ask* for a multikey index. You create an ordinary index, and if the field holds an **array** in any document, MongoDB marks the index `isMultiKey: true` automatically.

7.4 • Text index

Word-level search inside strings.

Text – tokenise, stop words, stem

← → keys also work

```
db.posts.createIndex({ title: "text" })
```

POSTS

p1

title: "Learn Node.js and the Express router"

p2

title: "Running MongoDB in Docker"

p3

title: "Why is my Express app slow?"

TEXT INDEX – STEMMED TERM → RECORDID (+ WEIGHT)

A normal index on `title` can only match the *whole* string (or a prefix with an anchored regex). A **text index** breaks strings into words so you can search inside them.

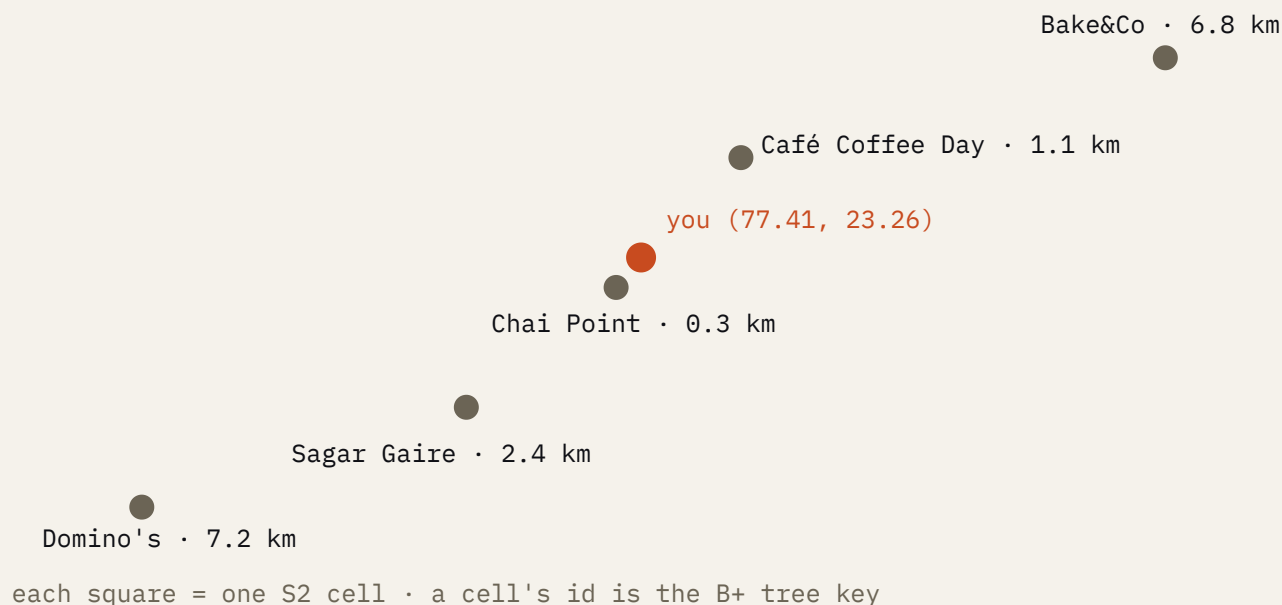
7.5 • Geospatial index (2dsphere)

'Cafés near me', answered with a B+ tree.

2dsphere – cells and expanding rings

← → keys also work

```
db.cafes.createIndex({ location: "2dsphere" })
```



Latitude/longitude are two numbers, but a B+ tree sorts one key. So how do you index 'near me'? The trick is to turn a 2-D position into a **single sortable number** such that nearby places get nearby numbers.

7.6 • Hashed index

Equality only — built for spreading shard keys evenly.

Hashed – random order on purpose

← → keys also work

```
db.users.createIndex({ email: "hashed" })
```

USERS

d1

d2

email: **riya@x.com**

email: **aman@x.com**

d3

email: **neha@x.com**

d4

email: **karan@x.com**

d5

email: **pooja@x.com**

d6

email: **rahul@x.com**

HASHED INDEX { EMAIL: "HASHED" } — SORTED BY HASH, NOT BY VALUE

A hashed index stores **hash(value)** instead of the value. Same B+ tree underneath — the key is just a 64-bit number produced from the field.

7.7 • Unique index

An ordinary index that refuses duplicate keys at write time.

Unique — E11000 in action

← → keys also work

```
db.users.createIndex({ email: 1 }, { unique: true })
```

USERS (+ INCOMING INSERT)

d1

email: **riya@x.com**

d2

email: **aman@x.com**

d3

email: **neha@x.com**

UNIQUE INDEX { EMAIL: 1 }

aman@x.com → **d2**

neha@x.com → d3

riya@x.com → d1

unique is a **property**, not a separate structure: an ordinary B+ tree index with one extra rule enforced on write — no two entries may share a key.

7.8 • Sparse index

Skip documents that do not have the field.

Sparse — smaller tree, one blind spot

← → keys also work

```
db.users.createIndex({ phone: 1 }, { sparse: true })
```

USERS — HALF HAVE NO PHONE

d1

name: **riya**
phone: 98...01

d2

name: **aman**
phone: — missing —

d3

name: **neha**
phone: 97...22

d4

name: **karan**
phone: — missing —

d5

name: **pooja**
phone: 96...33

d6

name: **rahul**
phone: — missing —

SPARSE INDEX { PHONE: 1 } — 3 ENTRIES, NOT 6

A normal index makes an entry for *every* document — documents without the field get a `null` key. If only 5% of users have a phone, 95% of the index is nulls: wasted space and slower writes.

7.9 • Partial index

Index only the documents you actually query.

Partial — filter at build time

← → keys also work

```
createIndex({ amount: 1 }, { partialFilterExpression: { status: "active" } })
```

ORDERS — MOST ARE FINISHED

o1
status: **active**
amount: **1200**

o2
status: **shipped**
amount: **800**

o3
status: **active**
amount: **450**

o4
status: **cancelled**
amount: **300**

o5
status: **active**
amount: **990**

o6
status: **delivered**
amount: **2100**

PARTIAL INDEX { AMOUNT: 1 } WHERE STATUS = "ACTIVE"

PLAN

—

Think of an e-commerce `orders` collection: 99% of orders are finished, but almost every dashboard query is about **active** ones. Indexing all orders wastes 99% of the tree on rows nobody queries.

7.10 • TTL index

Documents that delete themselves.

TTL – the background monitor

← → keys also work

```
db.sessions.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 })
```

SERVER CLOCK

10:10

TTL MONITOR

idle

EXPIRE AFTER

3600 s

SESSIONS

s1

createdAt: 09:00

s2

createdAt: 09:20

s3

createdAt: 09:45

s4

createdAt: 10:05

TTL INDEX { CREATEDAT: 1 } – OLDEST FIRST

09:00 → s1

09:20 → s2

09:45 → s3

10:05 → s4

Sessions, OTPs, password-reset tokens, logs — data that should *disappear* after a while. Instead of writing a cron job, put a **TTL** index on a date field with `expireAfterSeconds`. Here: sessions die one hour after `createdAt`.

7.11 • Wildcard index

Index every path under a prefix, for schemas you do not control.

Wildcard – path + value keys

← → keys also work

```
db.products.createIndex({ "attributes.$**": 1 })
```

PRODUCTS – ATTRIBUTES DIFFER PER PRODUCT

p1

attributes: { color: "red", size: "M" }

p2

attributes: { brand: "Nike", size: "L" }

p3

attributes: { color: "blue", material: "cotton" }

WILDCARD INDEX { "ATTRIBUTES.\$**": 1 } — KEY = PATH + VALUE

A product catalogue where every product has different attribute names: shirts have size and colour, phones have RAM and storage. You cannot create an index per attribute — you do not know them in advance. A **wildcard** index indexes *every* path under a prefix.

08 What to use when

A decision guide, the ESR rule, and the index you should not create.

You now know every type. The skill that pays is choosing — and, just as often, *not* adding an index. Start from the **queries**, never from the schema: an index that no query uses is pure cost.

Exact match on one field

Filter on A, sort by B

Filter + range + sort

Match inside an array

Search words in text

'Near me' / inside an area

Must be unique (maybe optional)

Only a small slice is ever queried

Auto-delete after N seconds

Unknown / user-defined fields

Shard key that must spread writes

Query runs once a month on a small collection

Single-field index

`createIndex({ email: 1 })`. Direction is irrelevant for a lone equality. If a compound index already starts with this field, you do not need this one.

The ESR rule, shown rather than told

Equality · Sort · Range – why the order matters

← → keys also work

```
db.orders.find({ status: "open", amount: { $gt: 500 } }).sort({ createdAt: -1 })
```

INDEX A – { STATUS: 1, AMOUNT: 1, CREATEDAT: -1 } (E · R · S)

closed · 11 Sep · ₹900 → o3

closed · 07 Sep · ₹2000 → o6

open · 10 Sep · ₹300 → o2

open · 08 Sep · ₹450 → o5

open · 09 Sep · ₹700 → o4

open · 12 Sep · ₹1200 → o1

INDEX A PLAN

—

INDEX B PLAN

—

One query, three kinds of predicate: an **Equality** (`status`), a **Range** (`amount > 500`) and a **Sort** (`createdAt`). All three fields will go in one compound index — the question is *in which order*.

How many indexes is too many?

There is a hard limit of **64 indexes per collection**, but you should be worried long before that. Each index is a full B+ tree that every insert, update and delete must maintain, and that competes for the same RAM cache as your data. A practical heuristic for a write-heavy collection is 'as few as the hot queries need, usually under ten'. Use `$indexStats` to find indexes with zero operations, and remember that a compound index makes any index on its prefix redundant.

DESIGN PROCEDURE THAT WORKS IN REAL PROJECTS

1. List the queries your app actually runs, with their filters, sorts and how often. 2. For each, write the fields as E / S / R. 3. Build the smallest set of compound indexes whose prefixes cover those shapes. 4. Verify each with `explain()`: IXSCAN, no SORT, docs examined $\approx$ returned. 5. Add unique/partial/TTL properties where the domain demands them. 6. Watch `$indexStats` in production and prune.

09 Pitfalls, performance and the write side

The mistakes that show up in code reviews, and how to spot them with `explain()`.

PITFALL	WHAT YOU SEE IN EXPLAIN()	FIX
Wrong compound order (range or sort field first)	<code>SORT</code> stage, or a COLLSCAN for the common query	Rebuild in ESR order
Index on a low-selectivity field (<code>isActive</code> , <code>gender</code>)	IXSCAN but <code>totalDocsExamined</code> $\approx$ half the collection	Put the selective field first, or make it a partial index on the rare value
Negation: <code>\$ne</code> , <code>\$nin</code> , <code>\$not</code>	index bounds <code>[MinKey, MaxKey]</code> — the index is walked entirely	Rewrite as a positive filter, or accept the scan
Unanchored regex <code>/gmail/</code>	full index scan	Anchor it: <code>/^ankur/</code> uses the index as a range; use a text index or Atlas Search for "contains"
Case-insensitive match <code>/^ankur/i</code>	full index scan despite the anchor	Store a lowercased copy of the field, or create the index with a collation (<code>strength: 2</code>) and query with the same collation
Type mismatch — querying a string field with a number	IXSCAN with zero results	MongoDB compares by type first; fix the data or the query
Sorting a huge result without an index	<code>SORT</code> + error "Sort exceeded memory limit of 100 MB"	Index the sort field (with any equality fields before it), or use <code>allowDiskUse</code> as a last resort
Too many overlapping indexes	long <code>rejectedPlans</code> , slow writes, big <code>totalIndexSize()</code>	Drop indexes that are prefixes of others; use <code>\$indexStats</code>
Trusting the planner blindly	a "wrong" index wins because of stale plan cache	<code>hint()</code> to test; plan cache clears on index changes

Index intersection: why it rarely saves you

MongoDB *can* use two single-field indexes for one query (an `AND_SORTED` / `AND_HASH` stage), by scanning both and intersecting the RecordIds. In practice the planner prefers one compound index almost every time, and for good reason: intersection reads two trees and then does set arithmetic, while a compound index does one seek. Do not build 'one index per field' and hope for intersection — build compound indexes for the query shapes you have.

The write side, honestly

Every index costs on three axes. **Write latency:** one more B+ tree insert (and possibly a page split) per index per write. **Disk:** indexes on big collections are often 20–40% of data size, and a text or multikey index can be

larger than the collection. **RAM:** a hot index that does not fit in cache turns every write into a random disk read to find the page to modify. Check `db.collection.stats().indexSizes` and keep your hot indexes in memory.

 **Mongoose autoIndex in production.** With `autoIndex: true` (the default), every deploy triggers `createIndex()` for every schema index. On an existing index this is a no-op, but a *changed* definition builds a new index on a live collection at startup, on every instance. Turn it off in production and create indexes as a deliberate migration step.

10 Interview questions, practice and cheat sheet

Close the loop: explain it back, then do it with your own hands.

Questions you will be asked

Q1 What is an index and why does it make reads faster? +

Q2 Why B+ tree and not a hash table or a binary search tree? +

Q3 What is the difference between a B-tree and a B+ tree? +

Q4 Walk me through inserting a key that causes a split. +

Q5 And deleting a key that causes underflow? +

Q6 What is the ESR rule? +

Q7 What is a covered query? +

Q8 Can MongoDB use an index for `find({ age: 22 })` if the only index is `{ city: 1, age: 1 }`? +

Q9 Sparse vs partial index? +

Q10 What does a multikey index do with an array, and what is its main restriction? +

Q11 When would you choose a hashed index? +

Q12 How do you find slow queries and unused indexes in production? +

Practice: see the numbers yourself

Do this in `mongosh` against a local MongoDB. It takes ten minutes and you will never forget the difference.

MONGOSH — BEFORE / AFTER LAB

Copy

```
// 1. Seed: 1 lakh users, random cities and ages
db.users.drop()
const cities = ["Bhopal","Indore","Pune","Delhi","Jaipur"]
const docs = []
for (let i = 0; i < 100000; i++) {
  docs.push({
    email: `user${i}@x.com`,
    city: cities[i % 5],
    age: 18 + (i * 7) % 40,
    createdAt: new Date(Date.now() - i * 60000),
  })
}
db.users.insertMany(docs)

// 2. Measure BEFORE — note executionTimeMillis, totalDocsExamined, stage
db.users.find({ email: "user77777@x.com" }).explain("executionStats")
db.users.find({ city: "Bhopal", age: { $gt: 30 } })
  .sort({ createdAt: -1 }).explain("executionStats")

// 3. Add indexes — decide the compound order with ESR before you type it
db.users.createIndex({ email: 1 }, { unique: true })
db.users.createIndex({ city: 1, createdAt: -1, age: 1 })

// 4. Measure AFTER — same two queries. Fill in the table in your notes.

// 5. Break it on purpose: try { age: 1, city: 1 } instead and explain again.
//    Then query by age only. Predict the plan before you run it.
```

QUERY	BEFORE: STAGE	BEFORE: DOCS EXAMINED	BEFORE: MS	AFTER: STAGE	AFTER: DOCS EXAMINED	AFTER: MS
email lookup						
city + age > 30, sort createdAt desc						
age only (with the wrong index)						

Cheat sheet

MONGOSH – EVERYTHING FROM THESE NOTES

Copy

```
db.c.createIndex({ f: 1 }) // single
db.c.createIndex({ a: 1, b: -1 }) // compound (ESR!)
db.c.createIndex({ tags: 1 }) // multikey (auto if array)
db.c.createIndex({ title: "text", body: "text" }) // text (one per collection)
db.c.createIndex({ loc: "2dsphere" }) // geo
db.c.createIndex({ _id: "hashed" }) // hashed (shard keys)
db.c.createIndex({ "attrs.$*": 1 }) // wildcard
db.c.createIndex({ email: 1 }, { unique: true }) // unique
db.c.createIndex({ phone: 1 }, { sparse: true }) // sparse
db.c.createIndex({ amount: 1 }, { partialFilterExpression: { status: "active" } })
db.c.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 }) // TTL
db.c.createIndex({ f: 1 }, { hidden: true }) // hidden
db.c.createIndex({ f: 1 }, { collation: { locale: "en", strength: 2 } }) // case-insensitive

db.c.getIndexes(); db.c.dropIndex("f_1"); db.c.hideIndex("f_1"); db.c.totalIndexSize()
db.c.find(q).sort(s).explain("executionStats") // IXSCAN? SORT? docs vs return
db.c.find(q).hint({ a: 1, b: -1 }) // force an index while testing
db.c.aggregate([ { $indexStats: {} } ]) // usage counts
db.runCommand({ collMod: "c", index: { keyPattern: { createdAt: 1 }, expireAfterSeconds: 7200 }
```

ONE PARAGRAPH TO REMEMBER

An index is a sorted B+ tree of keys pointing at documents, kept in sync on every write. Its shape matches the disk (one node = one page, hundreds of keys, 3–4 levels), so reads cost a few page hits instead of a

scan, and linked leaves make ranges and sorts free. Every index type is that same tree with a different key. Build indexes from your queries in ESR order, verify with `explain()`, and never forget that each one taxes every write.

Sheryians Coding School · Backend · MongoDB Indexing

Cohort notes · B+ tree demos computed live · MongoDB 7/8 semantics